

Foresight Fellowship 2027 - Christian Bucher Evidenzportfolio

Executive-Route und Evidenzzusammenfassung

Kernthese

Ich habe praktische Agenteninfrastruktur um Modelle herum gebaut: Remote-Auftragseingang, Dual-Brain-Verstehen und Bewertung, lokale Daemon-Orchestrierung, Modellrouting, Memory-Governance, Voice-Control und eine sauber abgegrenzte Forschungsrichtung.

- Secure Multiagent Systems aus Builder-Perspektive.
- Evidenz zuerst, klare Claim-Grenzen.
- Zukunftsprojekt: unsicherheitsbewusste KI für antimikrobielle Resistenz.

Reviewer-Pfad

- 5 Minuten: Remote/Dual-Brain, Miguel Core, Cascade Router.
- 20 Minuten: volles KI-System-Dossier und technische Appendix.
- Deep Review: ECSA-Zukunftsprojekt und einzelne PDF-Dossiers.

Claim-Disziplin

Das Paket trennt gebaute Prototypen, getestete Kernel, entworfene Projekte und spekulative Arbeit. Es vermeidet öffentliche lokale Pfade, Zugangsdaten, private Kanaldetails und unbestätigte medizinische/rechtliche/physikalische Claims.

Remote WhatsApp-to-Agent Command Loop

Ich wollte von unterwegs mit meinem Handy Arbeit an meinen Laptop übergeben koennen, ohne daraus eine gefaehrliche direkte Shell zu machen. Die WhatsApp-Nachricht sollte erst verstanden, in einen Arbeitsauftrag übersetzt, ausgeführt, bewertet und dann sauber zurückgemeldet werden.

- Das stärkste Signal ist nicht der Chatkanal, sondern die sichere Übersetzung von Alltagssprache in einen geprüften Arbeitsauftrag: Eingang, Interpretation, Ausführung, Bewertung und erst danach Rückmeldung.
- Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

Was ich mir dabei gedacht habe

Was ich mir dabei gedacht habe, war einfach, aber technisch anspruchsvoll: Ich wollte nicht am Laptop sitzen müssen und trotzdem sinnvoll Arbeit an ihn übergeben können, ohne mein Handy zu einer blinden Remote-Shell zu machen. Der interessante Teil war die Vertrauensgrenze. Eine Nachricht musste erst zum Arbeitsauftrag werden, ausgeführt und danach bewertet werden, bevor sie zu mir zurückkommt.

Was ich gebaut habe

- Ich habe Fernzugriff als Autoritäts- und Sicherheitsproblem behandelt, nicht nur als Komfortfunktion.
- Ich habe die Front-Agenten-Schicht von der tool-reichen Ausführungsschicht getrennt.
- Ich habe mobile Sprache/Chat in strukturierte Arbeit mit Erfolgskriterien übersetzt.

Mechanik im Detail

- Der wertvolle Teil ist nicht WhatsApp selbst. WhatsApp ist nur der reibungsarme Eingang. Der harte Teil ist die Übersetzung: eine lockere Nachricht vom Handy wird zu einem strukturierten Arbeitsauftrag mit Ziel, Komplexität, Modellhinweis und Erfolgskriterien.
- Das Dual-Brain-Muster trennt die Rolle, die versteht und bewertet, von der Rolle, die mit Tools ausführt. Dadurch wird eine Handy-Nachricht nicht zu einem ungeprüften Direktbefehl.
- Die Antwort kommt bewusst erst nach der Bewertung zurück. Das ist näher an Supervisor/Executor als an einem normalen Chatbot.

Miguel Core Unified Daemon

Ich wollte meine vielen einzelnen Agenten-, Voice-, Memory- und Tool-Skripte nicht als lose Sammlung behalten. Miguel Core war mein Versuch, daraus eine beobachtbare lokale Kontrollschicht zu machen.

- Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.
- Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

Was ich mir dabei gedacht habe

Miguel Core entstand aus einer sehr praktischen Frustration: Ich hatte zu viele starke Teile als einzelne Skripte und Services. Ich wollte daraus eine lokale Kontrollschicht machen, die beobachtbar, routbar und verständlicher ist.

Was ich gebaut habe

- Ich denke Agenten nicht als einzelne Prompts, sondern als Betriebssystem-nahe Services.
- Ich kann verstreute Prototypen in eine zentrale Kontrollfläche zusammenführen.
- Ich habe früh über Health, State, Tool-Oberflächen und Autonomiegrenzen nachgedacht.

Mechanik im Detail

- Miguel Core lässt sich am stärksten als lokale Agenten-Betriebsschicht erklären. Chat, Streaming, Voice, State, Memory, Prediction, Modellrouting, Orchestrierung und Tool-Endpunkte werden in eine beobachtbare Kontrollschicht gezogen.
- Die Route Surface ist wichtig, weil sie zeigt, dass die Arbeit von Einzelskripten in Richtung System mit Health, State, Conversation Persistence und Worker Delegation gegangen ist.
- Startup- und Handover-Dokumente gehören zur Evidenz: Sie definieren Boot Checks, Betriebsmodi, Quality Anchors, Autonomy Rings und State Probes.

Multi-Model Cascade Router

Ich wollte nicht alles blind durch ein Modell schicken. Der Router behandelt Modelle als Ressourcen mit Stärken, Kosten, Quoten, Ausfällen, Qualität und Datenschutzkontext.

- Zeigt Modell-Governance in der Praxis: Modelle werden nach Kosten, Qualität, Privacy, Quoten, Ausfallrisiko und Eskalationslogik gewählt, nicht nach Bauchgefühl.
- Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

Was ich mir dabei gedacht habe

Ich wollte Modellwahl nicht als Geschmack behandeln. Routing sollte Teil der Architektur sein: günstig, wenn möglich; stark, wenn nötig; lokal, wenn Privacy zählt; fallback-fähig, wenn ein Provider ausfällt.

Was ich gebaut habe

- Ich habe Modellwahl als Governance-System behandelt, nicht als Geschmacksfrage.
- Ich habe Kosten, Latenz, Quoten, Qualität und Fallbacks im selben System gedacht.
- Ich habe lokale und API-basierte Modelle in einer Routing-Logik verbunden.

Mechanik im Detail

- Der Router behandelt Modelle als Infrastrukturre Ressourcen mit Stärken, Kosten, Quoten, Cooldowns, Privacy-Profilen und Ausfallmodi.
- Die Task-Strategie entscheidet, ob Geschwindigkeit, Qualität, kostenlose Modelle, lokale Ausführung, Code-Fähigkeit oder bezahlte Eskalation wichtiger ist.
- Der Sicherheitswert liegt in graceful degradation: Wenn ein Modell oder eine Ebene ausfällt, existiert eine Policy für den nächsten Weg.